



DOM Scraping Part 2: Now with Jaxer 1.0

We're finalizing our Jaxer 1.0 release, and I thought I'd go back to Paul's ['DOM Scraping' example](#) and see what Jaxer 1.0 has to offer.

A main new feature is Jaxer.Sandbox, which lets you open new server-side sandboxed 'windows' and load pages into them. You can GET a page from any url, or even POST some content and load back the response. You can control whether JavaScript is executed in the window, whether it honors meta refreshes, whether it loads synchronously or asynchronously, etc. And, most important, the window and its contents have no access to Jaxer and its APIs nor to your app — but your app has full access to it.

There's a lot of other new goodness in Jaxer 1.0, as well as the official released version of the Mozilla engine found in Firefox 3. So for example `getElementsByClassName` is natively implemented (see [John Resig's speed comparison](#)), in addition to the other Mozilla features such as built-in [XPath](#) functionality and a very robust [DOM feature set](#) — just what you need for some serious 'screen scraping', mashups, and content repurposing.

Let's see it in action. First, let's grab the same three pages Paul used and extract some choice content elements into our page before sending it to the browser. We'll reuse the same Jaxer.Sandbox instance three times, each time loading a different page into it and grabbing some content by id or by classname. The page is basically a single script block:

```
01. <html>
02.   <head>
03.     <title>DOM Scraping 1.0</title>
04.   </head>
05.   <body>
06.     <script runat="server">
07.       ...all code here...
08.     </script>
09.   </body>
10. </html>
```

containing the following server-side JavaScript:

```
01. var sandbox = new Jaxer.Sandbox();
02. var item, contents;
03.
04. sandbox.open("http://www.news.com/");
05. contents = sandbox.document.getElementById("mostPopStories");
06. addToPage("News.com Popular Stories", contents);
07.
08. sandbox.open("http://www.washingtonpost.com/");
09. contents = sandbox.document.getElementsByClassName("top-box-in");
10. addToPage("Washington Post", contents);
11.
12. sandbox.open("http://ajaxian.com");
13. contents = sandbox.document.getElementsByClassName("activitystream");
14. addToPage("Ajaxian", contents);
```

and a single helper function that adds the given contents from the sandbox to the current page:

```
01. function addToPage(title, contents)
02. {
03.   var div = document.createElement('div');
04.
05.   var h2 = document.createElement('h2');
06.   h2.innerHTML = title;
07.   div.appendChild(h2);
08.
09.   if (!(contents instanceof HTMLElement)) contents = contents[0];
10.   div.appendChild(document.importNode(contents, true));
11.
12.   document.body.appendChild(div);
13. }
```

Here's the resulting page:

Ajax, meet server.

Ever wish you could use your Ajax, DOM, and Javascript skills on the server? Now you can!

Download Jaxer for your own server now!

Ajax everywhere. **aptana Jaxer**

The screenshot shows a web browser window with three tabs: index0.html, index1.html, and index2.html. The active tab (index0.html) displays a page titled "News.com Popular Stories" with a list of five items, each with a link and a date of July 28, 2008. Below this is a section for "Washington Post" with a list of "Most Viewed Articles" and a "Most Popular" note. Further down is an "Ajaxian" section with a list of links. The browser's status bar at the bottom indicates "5 Errors". The "Firefox Preview Console" is open, showing five identical error messages: "Exception: timeDifference is not defined File: http://127.0.0.1:8000/repo/DOMScraping/index0.htm".

That's not too bad, but the timestamps on the News.com section don't look good, and there are a bunch of JavaScript errors on the browser from left-over JavaScript content that was in the original WashingtonPost.com page. We can quickly clean that up with another, tiny DOM helper function that removes any DOM NodeList you pass to it. And while we're at it we'll also remove the reference to the client-side Jaxer framework since we won't be needing it here. The script block becomes:

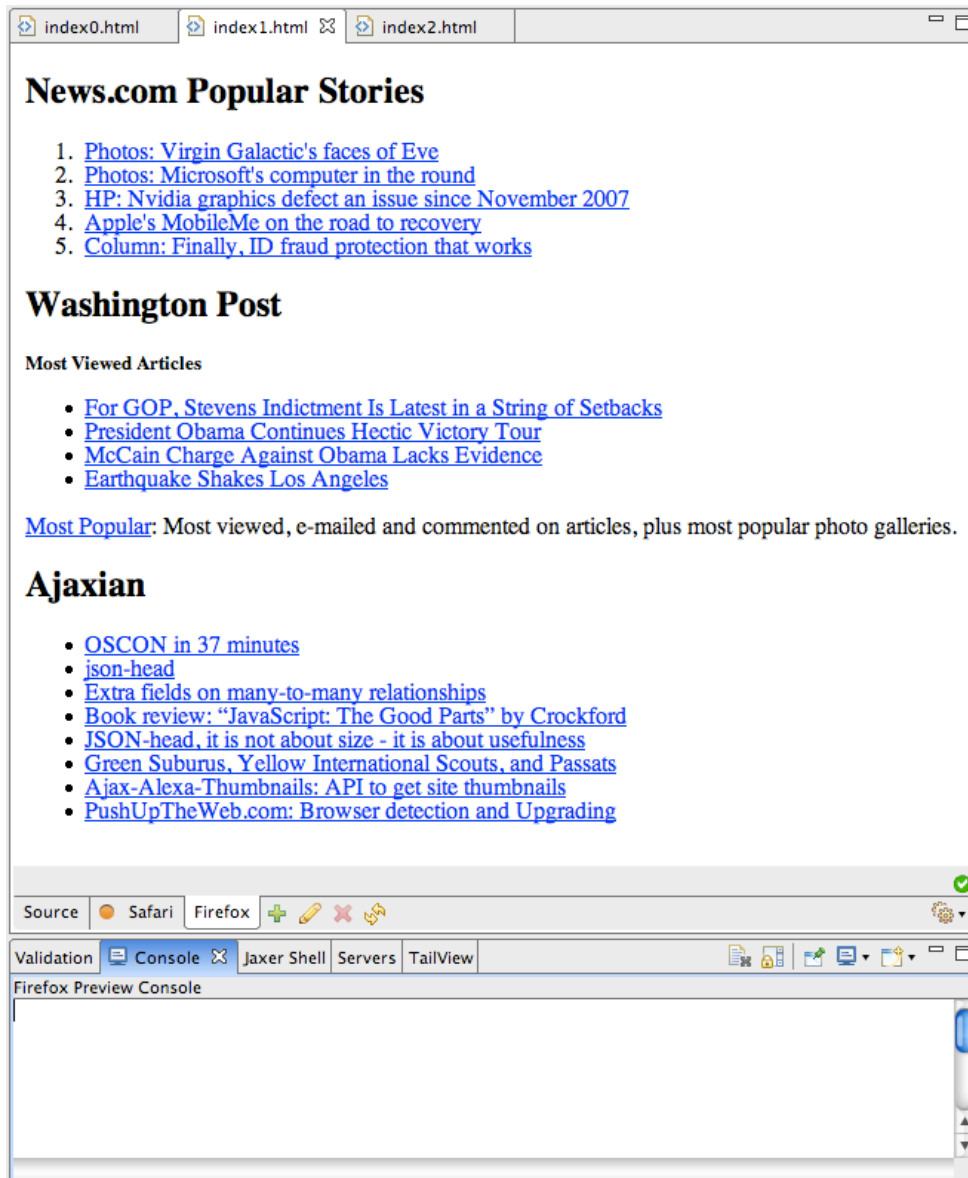
```
01. var sandbox = new Jaxer.Sandbox();
02. var item, contents, container;
03.
04. sandbox.open("http://www.news.com/");
05. contents = sandbox.document.getElementById("mostPopStories");
06. container = addToPage("News.com Popular Stories", contents);
07. // Remove the timestamps on each item
08. removeNodeList(container.getElementsByTagName("datestamp"));
09.
10. sandbox.open("http://www.washingtonpost.com/");
11. contents = sandbox.document.getElementsByTagName("top-box-in");
12. container = addToPage("Washington Post", contents);
13. // Remove the script tags so they don't appear on the client
14. removeNodeList(container.getElementsByTagName("script"));
15.
16. sandbox.open("http://ajaxian.com");
17. contents = sandbox.document.getElementsByTagName("activitystream");
18. addToPage("Ajaxian", contents);
```

```

19.
20. function addToPage(title, contents)
21. {
22.     var div = document.createElement('div');
23.
24.     var h2 = document.createElement('h2');
25.     h2.innerHTML = title;
26.     div.appendChild(h2);
27.
28.     if (!(contents instanceof HTMLElement)) contents = contents[0];
29.     div.appendChild(document.importNode(contents, true));
30.
31.     document.body.appendChild(div);
32.
33.     return div; // return the container so it can be further manipulated
34. }
35.
36. function removeNodeList(nodeList)
37. {
38.     while (nodeList.length > 0) nodeList[0].parentNode.removeChild(nodeList[0]);
39. }
40.
41. // We won't need Jaxer client-side:
42. Jaxer.response.setClientFramework(null);

```

Now that's much better:



But if you run this example yourself, you'll see it's slow: each of the original servers is a bit slow to load its entire page, and you have to load all three server-side before your page is ready to be sent to the browser. We could load all three in parallel, using the Jaxer.Sandbox's `async` capabilities and the `Jaxer.Thread.waitFor()` method to wait for them to load. But even then the user would see nothing in the browser until the slowest one was done loading server-side. So let's load them asynchronously client-side, by creating a server-side function called `getFragment` that loads a url and returns some of its contents, and setting `getFragment.proxy = true` so we can call it asynchronously from the client. The page now has two script blocks, one that runs on the browser and one on the server:

```

01. <html>
02.   <head>
03.     <title>DOM Scraping 1.0</title>
04.   </head>
05.   <body>
06.     <script>
07.       ...all client-side code here...
08.     </script>
09.     <script runat="server">
10.       ...all server-side code here...
11.     </script>

```

```

12. </body>
13. </html>

```

The client-side code is:

```

01. // On callback, append the results to this page
02. function appendHTML(html)
03. {
04.     document.body.innerHTML += html;
05. }
06. // onload is used to make sure getFragment is defined before it's called:
07. window.onload = function()
08. {
09.     Jaxer.Callback.TIMEOUT = 30 * 1000;
10.
11.     getFragment.async(appendHTML, "Ajaxian Activity Stream", "http://ajaxian.com", true, "activitystream");
12.
13.     getFragment.async(appendHTML, "News.com Popular Stories", "http://www.news.com/", false, "mostPopStories", "datestamp");
14.
15.     getFragment.async(appendHTML, "Washington Post", "http://www.washingtonpost.com/", true, "top-box-in");
16. }

```

and the server-side code is:

```

01. // Gets a fragment of the remote page's HTML, after some cleanup
02. function getFragment(title, url, isClassName, identifier, classesToRemove)
03. {
04.     var sandbox = new Jaxer.Sandbox(url);
05.     var contents = sandbox.document[isClassName ? 'getElementsByClassName' : 'getElementById'](identifier);
06.     var container = addToPage(title, contents);
07.     if (classesToRemove)
08.     {
09.         if (typeof classesToRemove == "string") classesToRemove = [classesToRemove];
10.         classesToRemove.forEach(function(className)
11.         {
12.             removeNodeList(container.getElementsByClassName(className));
13.         });
14.     }
15.     return container.innerHTML;
16. }
17. getFragment.proxy = true;
18.
19. function addToPage(title, contents)
20. {
21.     var div = document.createElement('div');
22.
23.     var h2 = document.createElement('h2');
24.     h2.innerHTML = title;
25.     div.appendChild(h2);
26.
27.     if (!(contents instanceof HTMLElement)) contents = contents[0];
28.     div.appendChild(document.importNode(contents, true));
29.
30.     document.body.appendChild(div);
31.
32.     return div;
33. }
34.
35. function removeNodeList(nodeList)
36. {
37.     while (nodeList.length > 0) nodeList[0].parentNode.removeChild(nodeList[0]);
38. }

```

The result looks the same as the previous screenshot, but the order in which the sections are displayed may vary, depending on which remote server responds first.